

# S2D-08: Dashboard Migration Best Practices

**Series:** S2D | **Notebook:** 8 of 9 | **Created:** January 2026 | **Last Updated:** 01/28/2026

## Overview

This notebook provides guidance for migrating Splunk dashboards to Dynatrace. While there's no automated conversion tool, understanding the mapping between Splunk and Dynatrace visualization concepts enables efficient manual migration.

## Prerequisites

| Requirement                  | Details                                               |
|------------------------------|-------------------------------------------------------|
| <b>Dynatrace Environment</b> | SaaS with Dashboards                                  |
| <b>Permissions</b>           | <code>documents.write</code> , <code>logs.read</code> |
| <b>Knowledge</b>             | Completed SPL to DQL translation (S2D-03)             |

## Learning Objectives

By the end of this notebook, you will be able to:

1. Map Splunk visualization types to Dynatrace equivalents
2. Convert Splunk dashboard structure to Dynatrace format
3. Apply best practices for dashboard organization
4. Use variables for interactive filtering

# Dashboard Structure Comparison

---

## Splunk Dashboard Components

| Component | Purpose                      |
|-----------|------------------------------|
| Dashboard | Container for panels         |
| Panel     | Individual visualization     |
| Search    | SPL query                    |
| Input     | User filter (dropdown, text) |
| Drilldown | Navigation action            |

## Dynatrace Dashboard Components

| Component | Purpose                  |
|-----------|--------------------------|
| Dashboard | Container for tiles      |
| Tile      | Individual visualization |
| DQL Query | Data source              |
| Variable  | User filter              |
| Drilldown | Navigation action        |

# Visualization Type Mapping

---

## Direct Equivalents

| Splunk       | Dynatrace    | Notes                    |
|--------------|--------------|--------------------------|
| Single Value | Single Value | Key metric display       |
| Line Chart   | Line Chart   | Timeseries visualization |
| Bar Chart    | Bar Chart    | Category comparison      |
| Pie Chart    | Pie Chart    | Distribution             |
| Table        | Table        | Tabular data             |
| Area Chart   | Area Chart   | Stacked timeseries       |
| Column Chart | Column Chart | Vertical bars            |

## Splunk-Specific Visualizations

| Splunk         | Dynatrace Alternative         |
|----------------|-------------------------------|
| Choropleth Map | Honeycomb (for entity groups) |
| Radial Gauge   | Single Value with thresholds  |
| Filler Gauge   | Single Value with thresholds  |
| Marker Gauge   | Single Value with thresholds  |
| Scatter Chart  | Not directly available        |

# Query Translation Examples

---

## Single Value Panel

```
// Single value: Total error count
// Splunk: index=app level=ERROR | stats count
fetch logs
| filter loglevel == "ERROR"
| summarize total_errors = count()
```

## Time-Series Line Chart

```
// Line chart: Error trend over time
// Splunk: index=app level=ERROR | timechart count
fetch logs
| filter loglevel == "ERROR"
| makeTimeseries error_count = count(), interval:5m
```

## Bar Chart by Category

```
// Bar chart: Errors by service
// Splunk: index=app level=ERROR | stats count by service
fetch logs
| filter loglevel == "ERROR"
| summarize error_count = count(), by:{k8s.deployment.name}
| sort error_count desc
| limit 10
```

## Table with Multiple Columns

```
// Table: Service health summary
// Splunk: index=app | stats count, count(eval(level="ERROR")) as errors by
service
fetch logs
| summarize
    total = count(),
    errors = countIf(loglevel == "ERROR"),
    warnings = countIf(loglevel == "WARN"),
    by:{k8s.deployment.name}
| fieldsAdd error_rate = round((toDouble(errors) / toDouble(total)) * 100,
decimals: 2)
| sort errors desc
```

## Using Variables for Filtering

### Defining Variables

Dynatrace dashboards support variables for interactive filtering:

| Variable Type | Use Case                    |
|---------------|-----------------------------|
| Text          | Free-form input             |
| Query-based   | Dynamic dropdown from query |
| Static        | Predefined options          |

### Using Variables in Queries

```
// Using dashboard variables in DQL
// Variables: $cluster, $namespace, $deployment
fetch logs
| filter matchesPhrase(k8s.cluster.name, $cluster)
| filter matchesPhrase(k8s.namespace.name, $namespace)
| filter matchesPhrase(k8s.deployment.name, $deployment)
| summarize count = count(), by:{loglevel}
```

## Variable-Populated Dropdown Query

```
// Query to populate a deployment dropdown variable
fetch logs
| filter isNotNull(k8s.deployment.name)
| summarize count(), by:{k8s.deployment.name}
| fields k8s.deployment.name
| sort k8s.deployment.name asc
```

## Dashboard Organization Best Practices

---

### Layout Recommendations

#### 1. Top Row: Key Metrics

- Single value tiles showing critical KPIs
- Use color thresholds for status indication

#### 2. Middle Section: Trends

- Time-series charts showing patterns over time
- Use consistent time intervals across related charts

#### 3. Bottom Section: Details

- Tables with detailed breakdown
- Log samples for investigation

### Naming Conventions

Follow the naming standards (see S2D-09):

- Dashboard: [AppName] Dashboard Title
- Example: [EasyTravel] Business Overview

## Log Searcher Dashboard Pattern

---

A common migration pattern is creating log searcher dashboards for investigating logs.

## VM Log Searcher

Variables: `host_name`, `log_source`

```
// VM Log Searcher - Log count by host
fetch logs
| filter matchesPhrase(host.name, $host_name)
| filter matchesPhrase(log.source, $log_source)
| summarize count = count(), by:{host.name}
| fields count, host.name
```

```
// VM Log Searcher - Log count by source
fetch logs
| filter matchesPhrase(host.name, $host_name)
| filter matchesPhrase(log.source, $log_source)
| summarize count = count(), by:{host.name, log.source}
| fields count, host.name, log.source
```

## Kubernetes Log Searcher

Variables: `k8s_cluster`, `k8s_namespace`, `k8s_deployment`

```
// K8s Log Searcher - Log count by cluster
fetch logs
| filter matchesPhrase(k8s.cluster.name, $k8s_cluster)
| filter matchesPhrase(k8s.namespace.name, $k8s_namespace)
| filter matchesPhrase(k8s.deployment.name, $k8s_deployment)
| summarize count = count(), by:{k8s.cluster.name}
| fields count, k8s.cluster.name
```

```
// K8s Log Searcher - Log count by deployment
fetch logs
| filter matchesPhrase(k8s.cluster.name, $k8s_cluster)
| filter matchesPhrase(k8s.namespace.name, $k8s_namespace)
| filter matchesPhrase(k8s.deployment.name, $k8s_deployment)
| summarize count = count(), by:{k8s.cluster.name, k8s.namespace.name,
k8s.deployment.name}
| fields count, k8s.cluster.name, k8s.namespace.name, k8s.deployment.name
```

# Migration Checklist

---

| Step | Action                                             |
|------|----------------------------------------------------|
| 1    | Inventory all Splunk dashboard panels              |
| 2    | Translate SPL queries to DQL (S2D-03)              |
| 3    | Create Dynatrace dashboard with appropriate layout |
| 4    | Add tiles with translated queries                  |
| 5    | Configure variables for filtering                  |
| 6    | Apply naming conventions                           |
| 7    | Validate data matches Splunk                       |
| 8    | Share with stakeholders                            |

## Next Steps

---

- **S2D-09: Naming Standards** - Apply consistent naming conventions

## References

---

- [Dynatrace Dashboards](#)
  - [Dashboard Variables](#)
  - [Visualization Types](#)
- 

*This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.*